

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (Tiếng Việt)

Aurélien Géron — bản dịch chỉnh sửa (2nd ed.)

Ngày 6 tháng 7 năm 2026

Mục lục

0.1	Đánh giá tốt hơn bằng cách sử dụng xác thực chéo	3
0.2	Tuning model của bạn	4
0.3	Tìm kiếm lưới	4
0.4	Tìm kiếm ngẫu nhiên	5
0.5	Phương pháp tập hợp	5
0.6	Phân tích các model tốt nhất và lỗi của chúng	5
0.7	Đánh giá hệ thống của bạn trên bộ thử nghiệm	5
1	Bài tập	8
1.1	Chương 3	9
1.2	Classification	9
1.3	Mnist	9
1.4	Training một classifier nhị phân	10
1.5	Các biện pháp thực hiện	10
1.6	Implementation riêng của API này, gọi là tf.keras, cung cấp hỗ trợ cho một số TensorFlow features nâng cao (ví dụ: khả năng tải dữ liệu hiệu quả).	10
1.7	Thực hiện xác thực chéo	11
1.8	Matrix nhầm lẫn	11
1.9	Đánh đổi độ chính xác/thu hồi	13
1.10	Đường cong ROC	14
1.11	Classification nhiều lớp	15
1.12	Phân tích lỗi	16
1.13	Classification đa nhãn	17
1.14	Classification đa đầu ra	18
2	Bài tập	18
2.1	Model training CHAPTER 4	19
2.2	Regression tuyến tính	20
2.3	Phương trình chuẩn	21
2.4	Độ phức tạp tính toán	22
2.5	Gradient Descent	22

Đầu tiên chúng ta nhập lớp `ColumnTransformer`, tiếp theo chúng ta lấy danh sách các tên cột bằng số và danh sách các tên cột classification, sau đó chúng ta xây dựng một `ColumnTransformer`. Hàm tạo yêu cầu một danh sách các bộ dữ liệu, trong đó mỗi bộ dữ liệu chứa một tên, một transformer và một danh sách tên (hoặc chỉ mục) của các cột mà bộ chuyển đổi sẽ được áp dụng. Trong ví dụ này, chúng ta chỉ định rằng các cột số sẽ được chuyển đổi bằng `num_pipeline` mà chúng ta đã xác định trước đó và các cột classification phải được chuyển đổi bằng `OneHotEncoder`. Cuối cùng, chúng ta áp dụng `ColumnTransformer` này cho dữ liệu vờ: nó áp dụng từng transformer cho các cột thích hợp và nối các đầu ra dọc theo trục thứ hai (máy biến áp phải trả về cùng số hàng).

Updated for

TensorFlow 2

Nếu bạn đang sử dụng `Scikit-Learn 0.19` trở về trước, bạn có thể sử dụng thư viện của bên thứ ba như `sklearn-pandas` hoặc bạn có thể tung ra transformer tùy chỉnh của riêng mình để có được chức năng tương tự như `ColumnTransformer`. Ngoài ra, bạn có thể sử dụng `FeatureUnion`

Lớp, có thể áp dụng các máy biến áp khác nhau và nối các đầu ra của chúng. Nhưng bạn không thể chỉ định các cột khác nhau cho mỗi transformer; tất cả đều áp dụng cho toàn bộ dữ liệu. Có thể khắc phục hạn chế này bằng cách sử dụng transformer tùy chỉnh để chọn cột (xem Jupyter notebook để biết ví dụ).

Chọn và training model

Cuối cùng! Bạn xác định vấn đề, lấy dữ liệu và khám phá nó, lấy mẫu training set và test set, đồng thời viết phép biến đổi pipelines để tự động dọn dẹp và chuẩn bị dữ liệu cho các thuật toán Machine Learning. Bây giờ bạn đã sẵn sàng để lựa chọn và training Machine Learning model.

Training và đánh giá trên training set

Tin vui là nhờ tất cả các bước trước đó, mọi việc giờ đây sẽ đơn giản hơn nhiều so với bạn nghĩ. Trước tiên hãy training Linear Regression model, giống như chúng ta đã làm trong chương trước:

Xong! Bây giờ bạn có Linear Regression model đang hoạt động. Hãy thử dùng thử trên một số phiên bản từ training set:

Nó hoạt động, mặc dù các dự đoán không chính xác lắm (ví dụ: dự đoán đầu tiên sai lệch gần 40%!). Hãy đo RMSE của regression model này trên toàn bộ tập training bằng cách sử dụng hàm `mean_squared_error()` của `Scikit-Learn`:

Điều này tốt hơn là không có gì, nhưng rõ ràng không phải là một điểm số cao: `median_housing_values` của hầu hết các quận đều nằm trong khoảng 120,000 and 265.000, do đó, lỗi dự đoán điển hình là 68.628 USD là không thỏa mãn lắm. Đây là một ví dụ về model underfitting dữ liệu training. Khi điều này xảy ra, điều đó có thể có nghĩa là features không cung cấp đủ

72 | Chương 2: Dự án Machine Learning toàn diện

Thông tin để đưa ra dự đoán tốt hoặc model không đủ mạnh. Như chúng ta đã thấy trong chương trước, các cách chính để khắc phục underfitting là chọn model mạnh hơn, cung cấp features tốt hơn cho thuật toán training hoặc giảm các ràng buộc trên model. Model này không được chuẩn hóa, điều này loại trừ tùy chọn cuối cùng. Bạn có thể thử thêm nhiều features hơn (ví dụ: nhật ký của quần thể), nhưng trước tiên hãy thử một model phức tạp hơn để xem nó hoạt động như thế nào.

Hãy training `DecisionTreeRegressor`. Đây là một model mạnh mẽ, có khả năng tìm kiếm các mối quan hệ phi tuyến phức tạp trong dữ liệu (`Decision Trees` được trình bày chi tiết hơn trong Chương 6). Code bây giờ trông quen thuộc:

Bây giờ model đã được training, hãy đánh giá nó trên training set:

Đợi đã, cái gì cơ!? Không có lỗi gì cả? Model này có thể thực sự hoàn hảo? Tất nhiên, nhiều khả năng là model đã khớp dữ liệu quá mức. Làm thế nào bạn có thể chắc chắn? Như chúng ta đã thấy trước đó, bạn không muốn chạm vào test set cho đến khi sẵn sàng khởi chạy model mà bạn tin tưởng, vì vậy bạn cần sử dụng một phần của training set để training và một phần của nó để xác thực model.

0.1 Đánh giá tốt hơn bằng cách sử dụng xác thực chéo

Một cách để đánh giá Decision Tree model là sử dụng hàm `train_test_split()` để chia training set thành training set nhỏ hơn và validation set, sau đó training models của bạn với training set nhỏ hơn và đánh giá chúng với validation set. Đó là một chút công việc, nhưng không có gì quá khó khăn và nó sẽ hoạt động khá tốt.

Một giải pháp thay thế tuyệt vời là sử dụng cross-validation feature K-fold của Scikit-Learn. Đoạn code sau chia ngẫu nhiên training set thành 10 tập hợp con riêng biệt được gọi là các nhóm, sau đó nó training và đánh giá Decision Tree model 10 lần, mỗi lần chọn một nhóm khác để đánh giá và training trên 9 nhóm còn lại. Kết quả là một mảng chứa 10 điểm đánh giá:

Hãy nhìn vào kết quả:

Bây giờ Decision Tree trông không còn đẹp như trước nữa. Trên thực tế, nó có vẻ hoạt động kém hơn Linear Regression model! Lưu ý rằng cross-validation không chỉ cho phép bạn nhận được ước tính về hiệu suất của model mà còn đo lường mức độ chính xác của ước tính này (tức là độ lệch chuẩn của nó). Decision Tree có số điểm xấp xỉ 71.407, thường là ± 2.439 . Bạn sẽ không có thông tin này nếu bạn chỉ sử dụng một validation set. Nhưng cross-validation phải trả phí training model nhiều lần, vì vậy không phải lúc nào cũng thực hiện được.

Hãy tính điểm tương tự cho Linear Regression model để chắc chắn:

Đúng vậy: Decision Tree model là overfitting tệ đến mức nó hoạt động kém hơn Linear Regression model.

Indexer: Judith McConville | Interior designer: David Futato | Cover designer: Karen Montgomery | Illustrator: Rebecca Demarest

74 | Chương 2: Dự án Machine Learning toàn diện

Ồ, cái này tốt hơn nhiều: Random Forests trông rất hứa hẹn. Tuy nhiên, hãy lưu ý rằng điểm trên training set vẫn thấp hơn nhiều so với các bộ xác thực, nghĩa là model vẫn là overfitting, training set. Các giải pháp khả thi cho overfitting là đơn giản hóa model, hạn chế nó (tức là chuẩn hóa nó) hoặc nhận thêm nhiều dữ liệu training. Tuy nhiên, trước khi tìm hiểu sâu hơn về Random Forests, bạn nên thử nhiều models khác từ nhiều loại thuật toán Machine Learning khác nhau (ví dụ: một số Máy Vector hỗ trợ với các hạt nhân khác nhau và có thể là mạng thần kinh), mà không tốn quá nhiều thời gian để điều chỉnh hyperparameters. Mục tiêu là đưa vào danh sách rút gọn một vài (hai đến năm) models đầy triển vọng.

Bạn nên lưu mọi model mà bạn thử nghiệm để có thể dễ dàng quay lại bất kỳ model nào bạn muốn. Đảm bảo bạn lưu cả hyperparameters và parameters đã training, cũng như điểm số cross-validation và có lẽ cả những dự đoán thực tế. Điều này sẽ cho phép bạn dễ dàng so sánh điểm số giữa các loại model và so sánh các loại lỗi mà chúng mắc phải. Bạn có thể dễ dàng lưu Scikit-Learn models bằng cách sử dụng mô-đun `dump` của Python hoặc bằng cách sử dụng thư viện `joblib`, hiệu quả hơn trong việc tuần tự hóa các mảng NumPy lớn (bạn có thể cài đặt thư viện này bằng `pip`):

0.2 Tuning model của bạn

Giả sử rằng bây giờ bạn có một danh sách rút gọn models đầy hứa hẹn. Bây giờ bạn cần tuning chúng. Hãy xem xét một số cách bạn có thể làm điều đó.

0.3 Tìm kiếm lưới

Một tùy chọn là thử nghiệm hyperparameters theo cách thủ công cho đến khi bạn tìm thấy sự kết hợp tuyệt vời giữa các giá trị hyperparameter. Đây sẽ là công việc rất tẻ nhạt và bạn có thể không có thời gian để khám phá nhiều sự kết hợp.

Thay vào đó, bạn nên lấy GridSearchCV của Scikit-Learn để tìm kiếm bạn. Tất cả những gì bạn cần làm là cho nó biết hyperparameters nào bạn muốn nó thử nghiệm và những giá trị nào cần thử, sau đó nó sẽ sử dụng cross-validation để đánh giá tất cả các kết hợp có thể có của các giá trị hyperparameter. Ví dụ: đoạn code sau tìm kiếm sự kết hợp tốt nhất của các giá trị hyperparameter cho RandomForestRegressor:

Khi bạn không biết hyperparameter nên có giá trị gì, một cách tiếp cận đơn giản là thử lũy thừa liên tiếp của 10 (hoặc một số nhỏ hơn nếu bạn muốn tìm kiếm chi tiết hơn, như minh họa trong ví dụ này với n_estimators hyperparameter).

Param_grid này yêu cầu Scikit-Learn trước tiên đánh giá tất cả $3 \times 4 = 12$ kết hợp của các giá trị n_estimators và max_features hyperparameter được chỉ định trong lệnh đầu tiên (đừng lo lắng về ý nghĩa của những hyperparameters này bây giờ; chúng sẽ được giải thích trong Chương 7), sau đó thử tất cả $2 \times 3 = 6$ kết hợp giá trị hyperparameter trong lệnh thứ hai, nhưng lần này là với bootstrap hyperparameter được đặt thành Sai thay vì Đúng (là giá trị mặc định cho hyperparameter này).

Grid search sẽ khám phá $12 + 6 = 18$ kết hợp các giá trị RandomForestRegressor hyperparameter và nó sẽ training mỗi model 5 lần (vì chúng ta đang sử dụng cross validation gấp năm lần). Nói cách khác, tổng cộng sẽ có $18 \times 5 = 90$ vòng training! Có thể mất khá nhiều thời gian nhưng khi hoàn thành bạn có thể có được sự kết hợp parameters tốt nhất như thế này:

76 | Chương 2: Dự án Machine Learning toàn diện

Vì 8 và 30 là giá trị tối đa đã được đánh giá, có lẽ bạn nên thử tìm kiếm lại với giá trị cao hơn; điểm số có thể tiếp tục được cải thiện.

Bạn cũng có thể nhận trực tiếp estimator tốt nhất:

Và tất nhiên điểm đánh giá cũng có sẵn:

Trong ví dụ này, chúng ta thu được giải pháp tốt nhất bằng cách đặt siêu tham số max_features thành 8 và n_estimators hyperparameter thành 30. Điểm RMSE cho sự kết hợp này là 49.682, cao hơn một chút so với điểm bạn đạt được trước đó bằng cách sử dụng

Giá trị hyperparameter mặc định (là 50.182). Xin chúc mừng, bạn đã tuning thành công model tốt nhất của mình!

Đừng quên rằng bạn có thể coi một số bước chuẩn bị dữ liệu là hyperparameters. Ví dụ: grid search sẽ tự động tìm hiểu xem có nên thêm feature mà bạn không chắc chắn hay không (ví dụ: sử dụng add_bedrooms_per_room hyperparameter của CombinedAttributesAdder transformer của bạn). Tương tự, nó có thể được sử dụng để tự động tìm ra cách tốt nhất để xử lý các ngoại lệ, các tính năng bị thiếu, lựa chọn feature, v.v.

0.4 Tìm kiếm ngẫu nhiên

Cách tiếp cận grid search phù hợp khi bạn khám phá tương đối ít kết hợp, như trong ví dụ trước, nhưng khi không gian tìm kiếm hyperparameter lớn, bạn nên sử dụng RandomizedSearchCV để thay thế. Lớp này có thể được sử dụng theo cách tương tự như lớp GridSearchCV, nhưng thay vì thử tất cả các kết hợp có thể, nó đánh giá một số kết hợp ngẫu nhiên nhất định bằng cách chọn một giá trị ngẫu nhiên cho mỗi hyperparameter ở mỗi lần lặp. Cách tiếp cận này có hai lợi ích chính:

- Nếu bạn để tìm kiếm ngẫu nhiên chạy trong 1.000 lần lặp, phương pháp này sẽ khám phá 1.000 giá trị khác nhau cho mỗi hyperparameter (thay vì chỉ một vài giá trị cho mỗi hyperparameter với phương pháp grid search).
- Đơn giản bằng cách thiết lập số lần lặp, bạn có nhiều quyền kiểm soát hơn đối với ngân sách tính toán mà bạn muốn phân bổ cho tìm kiếm hyperparameter.

0.5 Phương pháp tập hợp

Một cách khác để tuning hệ thống của bạn là cố gắng kết hợp models hoạt động tốt nhất. Nhóm (hoặc “nhóm”) thường sẽ hoạt động tốt hơn cá nhân model giỏi nhất (giống như Random Forests hoạt động tốt hơn Decision Trees cá nhân mà họ dựa vào), đặc biệt nếu cá nhân models mắc các loại lỗi rất khác nhau. Chúng ta sẽ đề cập đến chủ đề này chi tiết hơn trong Chương 7.

0.6 Phân tích các model tốt nhất và lỗi của chúng

Bạn sẽ thường hiểu rõ hơn về vấn đề bằng cách kiểm tra models tốt nhất. Ví dụ: RandomForestRegressor có thể chỉ ra tầm quan trọng tương đối của từng thuộc tính để đưa ra dự đoán chính xác:

78 | Chương 2: Dự án Machine Learning toàn diện

Hãy hiển thị các điểm quan trọng này bên cạnh tên thuộc tính tương ứng của chúng:

Với thông tin này, bạn có thể muốn thử loại bỏ một số features ít hữu ích hơn (ví dụ: dường như chỉ có một danh mục ocean_proximity thực sự hữu ích, vì vậy bạn có thể thử loại bỏ các danh mục khác).

Bạn cũng nên xem xét các lỗi cụ thể mà hệ thống của bạn mắc phải, sau đó cố gắng hiểu lý do tại sao nó mắc phải chúng và điều gì có thể khắc phục vấn đề (thêm features bổ sung hoặc loại bỏ những lỗi không có thông tin, loại bỏ các giá trị ngoại lệ, v.v.).

0.7 Đánh giá hệ thống của bạn trên bộ thử nghiệm

Sau khi điều chỉnh models của bạn một thời gian, cuối cùng bạn cũng có một hệ thống hoạt động đủ tốt. Bây giờ là lúc để đánh giá model cuối cùng trên test set. Không có gì đặc biệt về quá trình này; chỉ cần lấy predictors và các nhãn từ test set của bạn, chạy full_pipeline của bạn để chuyển đổi dữ liệu (gọi biến đổi(), không phải fit_transform()—bạn không muốn khớp test set!) và đánh giá model cuối cùng trên test set:

```
Final_model = grid_search.best_estimator_
```

Trong một số trường hợp, ước tính điểm về lỗi khái quát hóa như vậy sẽ không đủ thuyết phục bạn khởi chạy: điều gì sẽ xảy ra nếu nó chỉ tốt hơn 0,1% so với model hiện đang được sản xuất? Bạn có thể muốn biết ước tính này chính xác đến mức nào. Đối với

điều này, bạn có thể tính khoảng tin cậy 95% cho lỗi tổng quát hóa bằng cách sử dụng `scipy.stats.t.interval()`:

Nếu bạn đã thực hiện nhiều hyperparameter tuning, hiệu suất thường sẽ kém hơn một chút so với kết quả bạn đo được bằng cross-validation (vì cuối cùng hệ thống của bạn đã được tuning để hoạt động tốt trên dữ liệu xác thực và có thể sẽ không hoạt động tốt trên datasets không xác định). Trong ví dụ này không phải như vậy, nhưng khi điều này xảy ra, bạn phải chống lại sự cám dỗ điều chỉnh hyperparameters để làm cho các con số trông đẹp mắt trên test set; những cải tiến sẽ khó có thể khái quát hóa cho dữ liệu mới.

Bây giờ đến giai đoạn chuẩn bị khởi động dự án: bạn cần trình bày giải pháp của mình (làm nổi bật những gì bạn đã học được, những gì hiệu quả và những gì không, những giả định nào đã được đưa ra và những hạn chế của hệ thống của bạn là gì), ghi lại mọi thứ và tạo những bài thuyết trình hay với hình ảnh trực quan rõ ràng và những tuyên bố dễ nhớ (ví dụ: “thu nhập trung bình là predictor số một của giá nhà đất”). Trong ví dụ về nhà ở ở California này, hiệu suất cuối cùng của hệ thống không tốt hơn so với ước tính về giá của các chuyên gia, thường chênh lệch khoảng 20%, nhưng vẫn có thể là một ý tưởng hay nếu triển khai hệ thống, đặc biệt nếu điều này giải phóng thời gian cho các chuyên gia để họ có thể làm những công việc thú vị và hiệu quả hơn.

Khởi chạy, giám sát và bảo trì hệ thống của bạn

Hoàn hảo, bạn đã được chấp thuận để khởi động! Bây giờ bạn cần chuẩn bị sẵn giải pháp của mình để đưa vào sản xuất (ví dụ: đánh bóng code, viết tài liệu và kiểm tra, v.v.). Sau đó, bạn có thể triển khai model vào môi trường sản xuất của mình. Một cách để thực hiện việc này là lưu Scikit-Learn model đã training (ví dụ: sử dụng joblib), bao gồm toàn bộ quá trình tiền xử lý và dự đoán pipeline, sau đó tải model đã training này vào trong môi trường sản xuất của bạn và sử dụng nó để đưa ra dự đoán bằng cách gọi phương thức `Predict()` của nó. Ví dụ: có thể model sẽ được sử dụng trong một trang web: user sẽ nhập một số dữ liệu

80 | Chương 2: Dự án Machine Learning toàn diện

Về một quận mới và nhập vào nút Ước tính Giá. Thao tác này sẽ gửi một truy vấn chứa dữ liệu đến máy chủ web, máy chủ này sẽ chuyển tiếp dữ liệu đó đến ứng dụng web của bạn và cuối cùng code của bạn sẽ chỉ gọi phương thức dự đoán() của model (bạn muốn tải model khi khởi động máy chủ, thay vì mỗi khi model được sử dụng). Ngoài ra, bạn có thể gói model trong một dịch vụ web chuyên dụng mà ứng dụng web của bạn có thể truy vấn thông qua REST API23 (xem Hình 2-17). Điều này giúp việc nâng cấp model của bạn lên phiên bản mới dễ dàng hơn mà không làm gián đoạn ứng dụng chính. Nó cũng đơn giản hóa việc mở rộng quy mô vì bạn có thể bắt đầu bao nhiêu dịch vụ web nếu cần và cân bằng tải các yêu cầu đến từ ứng dụng web của bạn trên các dịch vụ web này. Hơn nữa, nó cho phép ứng dụng web của bạn sử dụng bất kỳ ngôn ngữ nào, không chỉ Python.

Một chiến lược phổ biến khác là triển khai model của bạn trên đám mây, chẳng hạn như trên Google Cloud AI Platform (trước đây gọi là Google Cloud ML Engine): chỉ cần lưu model của bạn bằng joblib và tải nó lên Google Cloud Storage (GCS), sau đó truy cập Google Cloud AI Platform và tạo phiên bản model mới, trỏ nó tới tệp GCS. Thế thôi! Điều này cung cấp cho bạn một dịch vụ web đơn giản giúp bạn cân bằng tải và mở rộng quy mô. Nó nhận các yêu cầu JSON chứa dữ liệu đầu vào (ví dụ: của một quận) và trả về các phản hồi JSON chứa các dự đoán. Sau đó, bạn có thể sử dụng dịch vụ web này trong trang web của mình (hoặc bất kỳ môi trường sản xuất nào bạn đang sử dụng). Như chúng ta sẽ thấy trong Chương 19, việc triển khai TensorFlow models trên Nền tảng AI không khác nhiều so với việc triển khai Scikit-Learn models.

Nhưng việc triển khai không phải là kết thúc của câu chuyện. Bạn cũng cần viết code giám sát để kiểm tra hiệu suất trực tiếp của hệ thống theo định kỳ và kích hoạt cảnh báo khi hệ thống giảm. Đây có thể là một sự sụt giảm mạnh, có thể là do một thành phần nào đó trong cơ sở hạ tầng của bạn bị hỏng, nhưng hãy lưu ý rằng đó cũng có thể là một sự suy giảm nhẹ mà có thể dễ dàng không được chú ý trong một thời gian dài. Điều này khá phổ biến vì models có xu hướng “thối rữa” theo thời gian: thực sự, thế giới thay đổi, vì vậy nếu model được training với dữ liệu của năm ngoái, nó có thể không thích ứng với dữ liệu ngày nay.

Ngay cả một model được training để classification ảnh chó và mèo cũng có thể cần được training lại thường xuyên, không phải vì chó và mèo sẽ biến đổi qua đêm mà vì máy ảnh liên tục thay đổi, cùng với định dạng hình ảnh, độ sắc nét, độ sáng và tỷ lệ kích thước. Hơn nữa, mọi người có thể yêu thích các giống chó khác nhau vào năm tới hoặc họ có thể quyết định mặc cho thú cưng của mình những chiếc mũ nhỏ xíu—ai biết được?

Vì vậy, bạn cần theo dõi hiệu suất trực tiếp của model của mình. Nhưng làm sao bạn làm được điều đó? Vâng, nó phụ thuộc. Trong một số trường hợp, hiệu suất của model có thể được suy ra từ các số liệu tiếp theo. Ví dụ: nếu model của bạn là một phần của hệ thống đề xuất và nó gợi ý các sản phẩm mà user có thể quan tâm thì bạn có thể dễ dàng theo dõi số lượng sản phẩm được đề xuất bán ra mỗi ngày. Nếu con số này giảm xuống (so với các sản phẩm không được khuyến dùng), thì nghi phạm chính là model. Điều này có thể là do dữ liệu pipeline bị hỏng hoặc có lẽ model cần được training lại trên dữ liệu mới (như chúng ta sẽ thảo luận sau).

Tuy nhiên, không phải lúc nào cũng có thể xác định được hiệu suất của model mà không cần bất kỳ phân tích nào của con người. Ví dụ: giả sử bạn đã training một hình ảnh classification model (xem Chương 3) để phát hiện một số lỗi sản phẩm trên dây chuyền sản xuất. Làm cách nào bạn có thể nhận được cảnh báo nếu hiệu suất của model giảm trước khi hàng nghìn sản phẩm bị lỗi được chuyển đến khách hàng của bạn? Một giải pháp là gửi cho người đánh giá một mẫu của tất cả các bức ảnh mà model đã classification (đặc biệt là những bức ảnh mà model không chắc chắn lắm). Tùy thuộc vào nhiệm vụ, người đánh giá có thể cần phải là chuyên gia hoặc họ có thể là những người không chuyên, chẳng hạn như nhân viên trên nền tảng cung cấp dịch vụ cộng đồng (ví dụ: Amazon Mechanical Turk). Trong một số ứng dụng, họ thậm chí có thể chính là user, phản hồi chẳng hạn như thông qua các cuộc khảo sát hoặc hình ảnh xác thực được sử dụng lại.²⁴

Dù bằng cách nào, bạn cần thiết lập một hệ thống giám sát (có hoặc không có người đánh giá con người để đánh giá model trực tiếp), cũng như tất cả các quy trình liên quan để xác định những việc cần làm trong trường hợp thất bại và cách chuẩn bị cho chúng. Thật không may, điều này có thể tốn rất nhiều công sức. Trên thực tế, việc này thường tốn nhiều công sức hơn việc xây dựng và training model.

Nếu dữ liệu tiếp tục phát triển, bạn sẽ cần cập nhật datasets của mình và training lại model của mình thường xuyên. Có lẽ bạn nên tự động hóa toàn bộ quá trình càng nhiều càng tốt. Dưới đây là một số điều bạn có thể tự động hóa:

- Thường xuyên thu thập dữ liệu mới và dán nhãn cho dữ liệu đó (ví dụ: sử dụng người đánh giá).
- Viết tập lệnh để training model và tuning hyperparameters một cách tự động. Tập lệnh này có thể chạy tự động, chẳng hạn hàng ngày hoặc hàng tuần, tùy thuộc vào nhu cầu của bạn.

82 | Chương 2: Dự án Machine Learning toàn diện

- Viết một tập lệnh khác sẽ đánh giá cả model mới và model trước đó trên test set đã cập nhật và triển khai model vào sản xuất nếu hiệu suất không giảm (nếu có, hãy đảm

bảo bạn điều tra lý do).

Bạn cũng nên đảm bảo đánh giá chất lượng dữ liệu đầu vào của model. Đôi khi hiệu suất sẽ giảm nhẹ do tín hiệu chất lượng kém (ví dụ: cảm biến trực trực gửi các giá trị ngẫu nhiên hoặc đầu ra của nhóm khác trở nên cũ), nhưng có thể mất một thời gian trước khi hiệu suất hệ thống của bạn suy giảm đủ để kích hoạt cảnh báo. Nếu bạn theo dõi đầu vào của model, bạn có thể nắm bắt được điều này sớm hơn. Ví dụ: bạn có thể kích hoạt cảnh báo nếu ngày càng có nhiều đầu vào thiếu feature hoặc nếu giá trị trung bình hoặc độ lệch chuẩn của nó chênh lệch quá xa so với training set hoặc feature được phân loại bắt đầu chứa các danh mục mới.

Cuối cùng, hãy đảm bảo bạn luôn sao lưu mọi model mà bạn tạo và có sẵn quy trình cũng như công cụ để nhanh chóng quay lại model trước đó, để phòng trường hợp model mới bắt đầu gặp lỗi nghiêm trọng vì lý do nào đó. Việc có bản sao lưu cũng giúp bạn có thể dễ dàng so sánh models mới với models trước đó. Tương tự, bạn nên sao lưu mọi phiên bản datasets của mình để có thể quay lại dataset trước đó nếu phiên bản mới bị hỏng (ví dụ: nếu dữ liệu mới được thêm vào hóa ra có nhiều dữ liệu ngoại lệ). Việc có bản sao lưu datasets của bạn cũng cho phép bạn đánh giá bất kỳ model nào so với bất kỳ dataset nào trước đó.

Bạn có thể muốn tạo một số tập hợp con của test set để đánh giá model của bạn hoạt động tốt như thế nào trên các phần cụ thể của dữ liệu. Ví dụ: bạn có thể muốn có một tập hợp con chỉ chứa dữ liệu gần đây nhất hoặc test set cho các loại đầu vào cụ thể (ví dụ: các quận nằm trong đất liền so với các quận nằm gần biển). Điều này sẽ giúp bạn hiểu sâu hơn về điểm mạnh và điểm yếu của model.

Như bạn có thể thấy, Machine Learning liên quan đến khá nhiều cơ sở hạ tầng, vì vậy đừng ngạc nhiên nếu dự án ML đầu tiên của bạn tốn nhiều công sức và thời gian để xây dựng và triển khai vào sản xuất. May mắn thay, một khi tất cả cơ sở hạ tầng đã sẵn sàng, việc đi từ ý tưởng đến sản xuất sẽ nhanh hơn nhiều.

Hãy thử nó!

Hy vọng rằng chương này đã cho bạn ý tưởng hay về dự án Machine Learning trông như thế nào cũng như chỉ cho bạn một số công cụ bạn có thể sử dụng để training một hệ thống tuyệt vời. Như bạn có thể thấy, phần lớn công việc nằm ở bước chuẩn bị dữ liệu: xây dựng các công cụ giám sát, thiết lập đánh giá con người pipelines và tự động hóa quá trình training model thường xuyên. Tất nhiên, các thuật toán Machine Learning rất quan trọng, nhưng có lẽ nó được ưa chuộng hơn.

Hãy thử nó! | 83

Bạn có thể cảm thấy thoải mái với quy trình tổng thể và biết rõ ba hoặc bốn thuật toán thay vì dành toàn bộ thời gian để khám phá các thuật toán nâng cao.

Vì vậy, nếu bạn chưa làm như vậy thì bây giờ là thời điểm tốt để chọn một chiếc máy tính xách tay, chọn một chiếc dataset mà bạn quan tâm và cố gắng thực hiện toàn bộ quá trình từ A đến Z. Một nơi tốt để bắt đầu là trên một trang web cạnh tranh như <http://kaggle.com/>, bạn sẽ có dataset để chơi cùng, một mục tiêu rõ ràng và mọi người để chia sẻ trải nghiệm. Chúc vui vẻ!

1 Bài tập

Các bài tập sau đây đều dựa trên dataset của chương này:

1. Hãy thử một bộ regression Support Vector Machine (`sklearn.svm.SVR`) với nhiều siêu parameters khác nhau, chẳng hạn như `kernel="tuyến tính"` (với nhiều giá trị khác

nhau cho siêu tham số C) hoặc `kernel="rbf"` (với nhiều giá trị khác nhau cho C và γ hyperparameters). Đừng lo lắng về ý nghĩa của những hyperparameters này. SVR predictor tốt nhất hoạt động như thế nào?

2. Hãy thử thay thế `GridSearchCV` bằng `RandomizedSearchCV`.

3. Thử thêm transformer vào pipeline chuẩn bị để chỉ chọn các thuộc tính quan trọng nhất.

4. Thử tạo một pipeline duy nhất thực hiện việc chuẩn bị dữ liệu đầy đủ cùng với dự đoán cuối cùng.

5. Tự động khám phá một số tùy chọn chuẩn bị bằng `GridSearchCV`.

Bạn có thể tìm thấy giải pháp cho những bài tập này trong Jupyter notebooks tại <https://github.com/ageron/handson-ml2>.

84 | Chương 2: Dự án Machine Learning toàn diện

1.1 Chương 3

1.2 Classification

Trong Chương 1 tôi đã đề cập rằng các tác vụ supervised learning phổ biến nhất là regression (dự đoán giá trị) và classification (dự đoán các lớp). Trong Chương 2, chúng ta đã khám phá nhiệm vụ regression, dự đoán giá trị nhà ở, sử dụng các thuật toán khác nhau như Linear Regression, Decision Trees và Random Forests (sẽ được giải thích chi tiết hơn trong các chương sau). Bây giờ chúng ta sẽ chuyển hệ thống attention của mình thành classification.

1.3 Mnist

Trong chương này, chúng ta sẽ sử dụng MNIST dataset, một bộ gồm 70.000 hình ảnh nhỏ gồm các chữ số được viết tay bởi học sinh trung học và nhân viên của Cục điều tra dân số US. Mỗi hình ảnh được gắn nhãn với chữ số mà nó đại diện. Bộ này đã được nghiên cứu nhiều đến mức nó thường được gọi là “hello world” của Machine Learning: bất cứ khi nào mọi người nghĩ ra một thuật toán classification mới, họ đều tò mò muốn xem nó sẽ hoạt động như thế nào trên MNIST và bất kỳ ai tìm hiểu Machine Learning sẽ xử lý dataset này sớm hay muộn.

Scikit-Learn cung cấp nhiều chức năng trợ giúp để tải xuống datasets phổ biến. MNIST là một trong số đó. Đoạn code sau tìm nạp MNIST dataset:

Datasets được tải bởi Scikit-Learn thường có cấu trúc từ điển tương tự, bao gồm:

- Khóa `DESCR` mô tả dataset
- Khóa dữ liệu chứa một mảng có một hàng cho mỗi phiên bản và một cột cho mỗi feature
- Khóa mục tiêu chứa một mảng có nhãn

Hãy nhìn vào các mảng này:

Có 70.000 hình ảnh và mỗi hình ảnh có 784 features. Điều này là do mỗi hình ảnh có kích thước 28×28 pixel và mỗi feature chỉ đại diện cho cường độ của một pixel, từ 0 (trắng) đến 255 (đen). Chúng ta hãy xem qua một chữ số từ dataset. Tất cả những gì bạn cần làm là lấy vector feature của một phiên bản, định hình lại nó thành một mảng 28×28 và hiển thị nó bằng hàm `imshow()` của Matplotlib:

Con số này trông giống như số 5 và thực sự đó là những gì nhãn hiệu cho chúng ta biết:

Lưu ý rằng nhãn là một chuỗi. Hầu hết các thuật toán ML đều mong đợi các số, vì vậy hãy chuyển y thành số nguyên:

86 | Chương 3: Classification

Để giúp bạn cảm nhận được độ phức tạp của nhiệm vụ classification, Hình 3-1 hiển thị thêm một vài hình ảnh từ MNIST dataset.

Nhưng chờ đã! Bạn phải luôn tạo test set và đặt nó sang một bên trước khi kiểm tra kỹ dữ liệu. MNIST dataset thực tế đã được chia thành training set (60.000 hình ảnh đầu tiên) và test set (10.000 hình ảnh cuối cùng):

Training set đã được xáo trộn đối với chúng ta, điều này tốt vì điều này đảm bảo rằng tất cả các nếp gấp cross-validation sẽ giống nhau (bạn không muốn một nếp gấp bị thiếu một số chữ số). Hơn nữa, một số thuật toán học rất nhạy cảm với thứ tự của các trường hợp training và chúng hoạt động kém nếu có nhiều trường hợp tương tự liên tiếp. Việc xáo trộn dataset đảm bảo điều này sẽ không xảy ra.²

1.4 Training một classifier nhị phân

Bây giờ hãy đơn giản hóa vấn đề và chỉ cố gắng xác định một chữ số—ví dụ: số 5. “Bộ dò 5” này sẽ là một ví dụ về classifier nhị phân, có khả năng phân biệt chỉ hai lớp, 5 và không phải 5. Hãy tạo các vector mục tiêu cho nhiệm vụ classification này:

Bây giờ hãy chọn một classifier và training nó. Một nơi tốt để bắt đầu là sử dụng classifier Giảm dần gradient ngẫu nhiên (SGD), sử dụng lớp SGDClassifier của Scikit-Learn. Classifier này có ưu điểm là có khả năng xử lý datasets rất lớn một cách hiệu quả. Điều này một phần là do SGD xử lý từng trường hợp training một cách độc lập (điều này cũng làm cho SGD rất phù hợp cho việc học trực tuyến), như chúng ta sẽ thấy sau. Hãy tạo một SGDClassifier và training nó trên toàn bộ training set:

SGDClassifier dựa vào tính ngẫu nhiên trong quá trình training (do đó có tên là “ngẫu nhiên”). Nếu bạn muốn có kết quả có thể lặp lại, bạn nên đặt random_state parameter.

Bây giờ chúng ta có thể sử dụng nó để phát hiện hình ảnh của số 5:

1.5 Các biện pháp thực hiện

Việc đánh giá một classifier thường phức tạp hơn đáng kể so với việc đánh giá một bộ regression, vì vậy chúng ta sẽ dành phần lớn thời gian của chương này cho chủ đề này. Có rất nhiều thước đo hiệu suất có sẵn, vì vậy hãy lấy một ly cà phê khác và sẵn sàng tìm hiểu nhiều khái niệm và từ viết tắt mới!

- Keras là Deep Learning API cấp cao giúp việc training và chạy neural networks rất đơn giản. Nó có thể chạy trên TensorFlow, Theano hoặc Microsoft Cognitive Toolkit (trước đây gọi là CNTK). TensorFlow đi kèm với

1.6 Implementation riêng của API này, gọi là tf.keras, cung cấp hỗ trợ cho một số TensorFlow features nâng cao (ví dụ: khả năng tải dữ liệu hiệu quả).

Một cách tốt để đánh giá model là sử dụng cross-validation, giống như bạn đã làm trong Chương 2.

1.7 Thực hiện xác thực chéo

Đôi khi, bạn sẽ cần kiểm soát nhiều hơn đối với quy trình cross-validation so với những gì Scikit-Learn cung cấp sẵn. Trong những trường hợp này, bạn có thể tự triển khai cross-validation. Đoạn code sau thực hiện gần giống như hàm `cross_val_score()` của Scikit-Learn và nó in ra kết quả tương tự:

Lớp `StratifiedKFold` thực hiện lấy mẫu phân tầng (như được giải thích trong Chương 2) để tạo ra các nếp gấp có tỷ lệ đại diện cho mỗi lớp. Ở mỗi lần lặp, code sẽ tạo một bản sao của classifier, training bản sao đó trên các phần training và đưa ra dự đoán trong phần thử nghiệm. Sau đó, nó đếm số lượng dự đoán đúng và đưa ra tỷ lệ dự đoán đúng.

Hãy sử dụng hàm `cross_val_score()` để đánh giá `SGDClassifier` model của chúng ta, sử dụng cross-validation gấp K với ba lần. Hãy nhớ rằng cross-validation gấp K có nghĩa là chia training set thành K nếp gấp (trong trường hợp này là ba), sau đó đưa ra dự đoán và đánh giá chúng trên mỗi nếp gấp bằng cách sử dụng model đã được training trên các nếp gấp còn lại (xem Chương 2):

Ồ! Độ chính xác trên 93% (tỷ lệ dự đoán đúng) trên tất cả các lần cross-validation? Điều này có vẻ tuyệt vời phải không? Chà, trước khi bạn quá phấn khích, hãy xem xét một classifier rất ngớ ngẩn chỉ classification từng hình ảnh trong lớp “không phải 5”:

Bạn có đoán được độ chính xác của model này không? Hãy cùng tìm hiểu:

Đúng vậy, nó có độ chính xác trên 90%! Điều này đơn giản là vì chỉ có khoảng 10% số hình ảnh là số 5, vì vậy nếu bạn luôn đoán rằng một hình ảnh không phải là số 5 thì bạn sẽ đúng khoảng 90%. Đánh bại Nostradamus.

Điều này chứng tỏ tại sao độ chính xác thường không phải là thước đo hiệu suất ưa thích cho các classifier, đặc biệt là khi bạn đang xử lý datasets bị lệch (tức là khi một số lớp xuất hiện thường xuyên hơn nhiều so với các lớp khác).

1.8 Matrix nhầm lẫn

Một cách tốt hơn nhiều để đánh giá hiệu suất của bộ được phân loại là xem xét matrix nhầm lẫn. Ý tưởng chung là đếm số lần các phiên bản của lớp A được phân loại thành lớp B. Ví dụ, để biết số lần classifier nhầm lẫn hình ảnh của 5 với 3, bạn sẽ nhìn vào hàng thứ năm và cột thứ ba của matrix nhầm lẫn.

Để tính toán confusion matrix, trước tiên bạn cần có một bộ dự đoán để có thể so sánh chúng với các mục tiêu thực tế. Bạn có thể đưa ra dự đoán về test set, nhưng bây giờ hãy giữ nguyên nó (hãy nhớ rằng bạn chỉ muốn sử dụng test set khi kết thúc dự án của mình, sau khi bạn đã có classifier mà bạn đã sẵn sàng khởi chạy). Thay vào đó, bạn có thể sử dụng hàm `cross_val_predict()`:

90 | Chương 3: Classification

Mỗi hàng trong confusion matrix đại diện cho một lớp thực tế, trong khi mỗi cột đại diện cho một lớp được dự đoán. Hàng đầu tiên của matrix này xem xét các hình ảnh không phải 5 (lớp âm): 53.057 trong số chúng được phân loại chính xác là không phải 5 (chúng được gọi là âm bản thực), trong khi 1.522 còn lại bị classification sai thành 5 (dương tính giả). Hàng thứ hai xem xét hình ảnh của 5s (lớp dương): 1.325 bị classification sai thành không phải 5 (âm tính giả), trong khi 4.096 còn lại được phân loại chính xác là 5s (dương tính thật). Một classifier hoàn hảo sẽ chỉ có các giá trị dương thực và âm thực, do đó confusion matrix của nó sẽ chỉ có các giá trị khác 0 trên đường chéo chính của nó (trên cùng bên trái đến dưới cùng bên phải):

Confusion matrix cung cấp cho bạn nhiều thông tin, nhưng đôi khi bạn có thể thích

số liệu ngắn gọn hơn. Một điều thú vị cần xem xét là tính chính xác của những dự đoán tích cực; đây được gọi là precision của classifier (Phương trình 3-1).

$$\text{Equation 3-1. Precision} = \frac{TP}{TP + FP}$$

TP là số lượng kết quả dương tính thật và FP là số lượng kết quả dương tính giả.

Một cách đơn giản để có precision hoàn hảo là đưa ra một dự đoán tích cực duy nhất và đảm bảo nó đúng (precision = 1/1 = 100%). Nhưng điều này sẽ không hữu ích lắm vì classifier sẽ bỏ qua tất cả ngoại trừ một trường hợp tích cực. Vì vậy, precision thường được sử dụng cùng với một số liệu khác có tên recall, còn được gọi là độ nhạy hoặc tỷ lệ dương thực sự (TPR): đây là tỷ lệ các trường hợp dương tính được classifier phát hiện chính xác (Phương trình 3-2).

$$\text{Equation 3-2. Recall} = \frac{TP}{TP + FN}$$

Tất nhiên, FN là số lượng âm tính giả.

Nếu bạn bối rối về confusion matrix, Hình 3-2 có thể hữu ích.

Precision và Recall

Scikit-Learn cung cấp một số hàm để tính toán các số liệu classification, bao gồm độ chính xác và recall:

Bây giờ máy dò 5 của bạn trông không còn sáng bóng như khi bạn nhìn vào độ chính xác của nó. Khi nó khẳng định một hình ảnh đại diện cho số 5 thì nó chỉ đúng 72,9%. Hơn nữa, nó chỉ phát hiện được 75,6% trong số 5s.

Việc kết hợp precision và recall thành một số liệu duy nhất gọi là điểm F1 thường rất thuận tiện, đặc biệt nếu bạn cần một cách đơn giản để so sánh hai classifier. Điểm F1 là giá trị trung bình hài hòa của precision và recall (Công thức 3-3). Trong khi giá trị trung bình thông thường xử lý tất cả các giá trị như nhau, thì giá trị trung bình hài hòa mang lại nhiều weight hơn cho các giá trị thấp. Do đó, classifier sẽ chỉ nhận được điểm F1 cao nếu cả recall và precision đều cao.

Phương trình 3-3. F1

$$2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} = F1$$

TP + FN + FP 2

92 | Chương 3: Classification

Để tính điểm F1, chỉ cần gọi hàm `f1_score()`:

Điểm F1 ưu tiên các classifier có precision và recall tương tự. Đây không phải lúc nào cũng là điều bạn muốn: trong một số bối cảnh, bạn chủ yếu quan tâm đến precision, và trong các bối cảnh khác, bạn thực sự quan tâm đến recall. Ví dụ: nếu bạn đã training một classifier để phát hiện các video an toàn cho trẻ em, thì có thể bạn sẽ thích một classifier loại bỏ nhiều video tốt (recall thấp) nhưng chỉ giữ lại những video an toàn (precision cao), hơn là một classifier có recall cao hơn nhiều nhưng lại để một số video thực sự tệ hiển thị trong sản phẩm của bạn (trong những trường hợp như vậy, bạn thậm chí có thể muốn thêm pipeline của con người để kiểm tra lớp- lựa chọn video của sifier). Mặt khác, giả sử bạn training một classifier để phát hiện những kẻ trộm đồ trong hình ảnh giám sát: có thể sẽ ổn nếu classifier của bạn chỉ có 30% precision miễn là nó có 99% recall (chắc chắn, nhân viên bảo vệ sẽ nhận được một vài cảnh báo sai, nhưng hầu hết tất cả những kẻ trộm đồ sẽ bị bắt).

Thật không may, bạn không thể có cả hai cách: tăng precision sẽ giảm recall và ngược lại. Đây được gọi là sự đánh đổi precision/recall.

1.9 Đánh đổi độ chính xác/thu hồi

Để hiểu sự cân bằng này, chúng ta hãy xem cách SGDClassifier đưa ra quyết định classification. Đối với mỗi trường hợp, nó tính điểm dựa trên hàm quyết định. Nếu điểm đó lớn hơn ngưỡng, nó sẽ gán phiên bản đó vào lớp tích cực; nếu không nó sẽ gán nó vào lớp phủ định. Hình 3-3 cho thấy một số chữ số được sắp xếp từ điểm thấp nhất ở bên trái đến điểm cao nhất ở bên phải. Giả sử ngưỡng quyết định được đặt ở mũi tên trung tâm (giữa hai số 5): bạn sẽ tìm thấy 4 kết quả dương tính thực (5 giây thực tế) ở bên phải ngưỡng đó và 1 kết quả dương tính giả (thực tế là 6). Do đó, với ngưỡng đó thì precision là 80% (4 trên 5). Nhưng trong số 6 giây thực tế, classifier chỉ phát hiện được 4, vì vậy recall là 67% (4 trên 6). Nếu bạn tăng ngưỡng (di chuyển nó sang mũi tên bên phải), dương tính giả (6) sẽ trở thành âm tính thực, do đó tăng precision (lên tới 100% trong trường hợp này), nhưng một dương tính thực sẽ trở thành âm tính giả, giảm recall xuống 50%. Ngược lại, việc hạ ngưỡng sẽ làm tăng recall và giảm precision.

Scikit-Learn không cho phép bạn đặt ngưỡng trực tiếp nhưng nó cung cấp cho bạn quyền truy cập vào các điểm quyết định mà nó sử dụng để đưa ra dự đoán. Thay vì gọi phương thức dự đoán() của classifier, bạn có thể gọi phương thức decision_function() của nó, phương thức này trả về điểm cho từng phiên bản và sau đó sử dụng bất kỳ ngưỡng nào bạn muốn để đưa ra dự đoán dựa trên những điểm số đó:

Điều này xác nhận rằng việc tăng ngưỡng sẽ làm giảm recall. Hình ảnh thực sự đại diện cho số 5 và classifier sẽ phát hiện ra nó khi ngưỡng bằng 0, nhưng nó sẽ bỏ lỡ khi ngưỡng được tăng lên 8.000.

Với những điểm số này, hãy sử dụng hàm precision_recall_curve() để tính precision và recall cho tất cả các ngưỡng có thể có:

94 | Chương 3: Classification

Cuối cùng, sử dụng Matplotlib để vẽ precision và recall dưới dạng các hàm của giá trị ngưỡng (Hình 3-4):

Bạn có thể thắc mắc tại sao đường cong precision lại gập ghềnh hơn đường cong recall trong Hình 3-4. Lý do là đôi khi precision có thể giảm khi bạn nâng ngưỡng lên (mặc dù nhìn chung nó sẽ tăng lên). Để hiểu lý do tại sao, hãy nhìn lại Hình 3-3 và để ý điều gì xảy ra khi bạn bắt đầu từ ngưỡng trung tâm và di chuyển nó chỉ một chữ số sang phải: precision đi từ 4/5 (80%) xuống 3/4 (75%). Mặt khác, recall chỉ có thể descent khi ngưỡng tăng lên, điều này giải thích tại sao đường cong của nó trông trơn tru.

Một cách khác để chọn một sự cân bằng precision/recall tốt là vẽ biểu đồ precision trực tiếp theo recall, như trong Hình 3-5 (ngưỡng tương tự như trước đó được đánh dấu).

Bạn có thể thấy precision thực sự bắt đầu giảm mạnh khoảng 80% recall. Có thể bạn sẽ muốn chọn mức cân bằng precision/recall ngay trước mức giảm đó—ví dụ: ở khoảng 60% recall. Nhưng tất nhiên, sự lựa chọn phụ thuộc vào dự án của bạn.

Giả sử bạn quyết định nhắm tới 90% precision. Bạn tra cứu ô đầu tiên và thấy rằng bạn cần sử dụng ngưỡng khoảng 8.000. Để chính xác hơn, bạn có thể tìm kiếm ngưỡng thấp nhất mang lại cho bạn ít nhất 90% precision (np.argmax() sẽ cung cấp cho bạn chỉ mục đầu tiên của giá trị tối đa, trong trường hợp này có nghĩa là giá trị True đầu tiên):

Để đưa ra dự đoán (hiện tại trên training set), thay vì gọi phương thức Predict() của classifier, bạn có thể chạy code này:

Hãy cùng kiểm tra những dự đoán này' precision và recall:

Tuyệt vời, bạn có classifier precision 90%! Như bạn có thể thấy, khá dễ dàng để tạo một classifier với hầu hết mọi precision mà bạn muốn: chỉ cần đặt một ngưỡng đủ cao là bạn đã hoàn tất. Nhưng chờ đã, không nhanh thế đâu. Classifier precision cao sẽ không

hữu ích lắm nếu recall của nó quá thấp!

96 | Chương 3: Classification

Nếu ai đó nói, “Hãy đạt precision 99%,” bạn nên hỏi, “Ở mức recall nào?”

1.10 Đường cong ROC

Đường cong đặc tính vận hành máy thu (ROC) là một công cụ phổ biến khác được sử dụng với các classifier nhị phân. Nó rất giống với đường cong precision/recall, nhưng thay vì vẽ biểu đồ precision so với recall, đường cong ROC vẽ tỷ lệ dương tính thực (tên gọi khác của recall) so với tỷ lệ dương tính giả (FPR). FPR là tỷ lệ các trường hợp tiêu cực được phân loại không chính xác thành tích cực. Nó bằng $1 - \text{tỷ lệ âm tính thực sự (TNR)}$, là tỷ lệ của các trường hợp âm tính được phân loại chính xác là âm tính. TNR còn được gọi là độ đặc hiệu. Do đó, đường cong ROC thể hiện độ nhạy (recall) so với $1 - \text{độ đặc hiệu}$.

Để vẽ đường cong ROC, trước tiên bạn sử dụng hàm `roc_curve()` để tính toán TPR và FPR cho các giá trị ngưỡng khác nhau:

Sau đó, bạn có thể vẽ đồ thị FPR với TPR bằng Matplotlib. Code này tạo ra âm mưu trong Hình 3-6:

Một cách để so sánh các bộ được phân loại là đo diện tích dưới đường cong (AUC). Một classifier hoàn hảo sẽ có ROC AUC bằng 1, trong khi một classifier hoàn toàn ngẫu nhiên sẽ có ROC AUC bằng 0,5. Scikit-Learn cung cấp chức năng tính toán ROC AUC:

Vì đường cong ROC rất giống với đường cong precision/recall (PR), bạn có thể thắc mắc làm thế nào để quyết định nên sử dụng đường cong nào. Theo nguyên tắc chung, bạn nên ưu tiên đường cong PR bất cứ khi nào loại dương tính hiếm gặp hoặc khi bạn quan tâm nhiều đến kết quả dương tính giả hơn là âm tính giả. Nếu không, hãy sử dụng đường cong ROC. Ví dụ: nhìn vào đường cong ROC trước đó (và điểm ROC AUC), bạn có thể nghĩ rằng classifier thực sự tốt. Nhưng điều này chủ yếu là do có ít điểm tích cực (5) so với điểm tiêu cực (không phải 5). Ngược lại, đường cong PR cho thấy rõ rằng classifier có chỗ cần cải thiện (đường cong có thể gần góc trên bên trái hơn).

Bây giờ chúng ta hãy training `RandomForestClassifier` và so sánh đường cong ROC và điểm số ROC AUC của nó với điểm số của `SGDClassifier`. Trước tiên, bạn cần lấy điểm cho từng phiên bản trong training set. Nhưng do cách thức hoạt động của nó (xem Chương 7), lớp `Random ForestClassifier` không có phương thức `decision_function()`. Thay vào đó, nó

98 | Chương 3: Classification

Có phương thức `predict_proba()`. Classifier Scikit-Learn thường có cái này hoặc cái kia hoặc cả hai. Phương thức `predict_proba()` trả về một mảng chứa một hàng cho mỗi thể hiện và một cột cho mỗi lớp, mỗi mảng chứa xác suất mà thể hiện đã cho thuộc về lớp đã cho (ví dụ: 70% khả năng hình ảnh đại diện cho số 5):

Bây giờ bạn đã sẵn sàng vẽ đường cong ROC. Việc vẽ đồ thị đường cong ROC đầu tiên cũng rất hữu ích để xem chúng so sánh như thế nào (Hình 3-7):

Như bạn có thể thấy trong Hình 3-7, đường cong ROC của `RandomForestClassifier` trông đẹp hơn nhiều so với đường cong của `SGDClassifier`: nó gần với góc trên cùng bên trái hơn nhiều. Kết quả là điểm ROC AUC của nó cũng tốt hơn đáng kể:

Hãy thử đo điểm precision và recall: bạn sẽ thấy 99,0% precision và 86,6% recall. Không quá tệ!

Bây giờ bạn đã biết cách training các classifier nhị phân, chọn số liệu thích hợp cho nhiệm vụ của mình, đánh giá các classifier bằng cách sử dụng cross-validation, chọn sự cân bằng precision/recall phù hợp với nhu cầu của bạn và sử dụng các đường cong ROC và điểm số ROC AUC để so sánh các models khác nhau. Bây giờ chúng ta hãy thử phát hiện nhiều hơn chỉ số 5.

1.11 Classification nhiều lớp

Trong khi các classifier nhị phân phân biệt giữa hai lớp, các classifier nhiều lớp (còn gọi là các classifier đa thức) có thể phân biệt giữa nhiều hơn hai lớp.

Một số thuật toán (chẳng hạn như classifier SGD, classifier Random Forest và classifier Bayes ngây thơ) có khả năng xử lý nguyên bản nhiều lớp. Những cái khác (chẳng hạn như classifier Logistic Regression hoặc Support Vector Machine) là các classifier nhị phân nghiêm ngặt. Tuy nhiên, có nhiều chiến lược khác nhau mà bạn có thể sử dụng để thực hiện classification đa lớp với nhiều classifier nhị phân.

Một cách để tạo ra một hệ thống có thể classification các hình ảnh chữ số thành 10 lớp (từ 0 đến 9) là training 10 classifier nhị phân, một classifier cho mỗi chữ số (một bộ dò 0, một bộ dò 1, một bộ dò 2, v.v.). Sau đó, khi bạn muốn classification một hình ảnh, bạn sẽ nhận được điểm quyết định từ mỗi classifier cho hình ảnh đó và bạn chọn lớp có classifier có điểm cao nhất. Đây được gọi là chiến lược một đấu với phần còn lại (OvR) (còn được gọi là chiến lược một đấu với tất cả).

Một chiến lược khác là training một classifier nhị phân cho mỗi cặp chữ số: một để phân biệt 0 và 1, một để phân biệt 0 và 2, một để phân biệt 1 và 2, v.v. Đây được gọi là chiến lược một chọi một (OvO). Nếu có N lớp, bạn cần training $N \times (N - 1) / 2$ classifier. Đối với bài toán MNIST, điều này có nghĩa là training 45 classifier nhị phân! Khi muốn classification một hình ảnh, bạn phải chạy hình ảnh đó qua tất cả 45 classifier và xem lớp nào thắng nhiều trận đấu tay đôi nhất. Ưu điểm chính của OvO là mỗi classifier chỉ cần được training về phần training set cho hai lớp mà nó phải phân biệt.

Một số thuật toán (chẳng hạn như classifier Support Vector Machine) có quy mô kém theo kích thước của training set. Đối với các thuật toán này, OvO được ưa thích hơn vì training nhiều classifier trên các tập training nhỏ nhanh hơn so với training một số classifier trên các tập training lớn. Tuy nhiên, đối với hầu hết các thuật toán classification nhị phân, OvR được ưu tiên hơn.

100 | Chương 3: Classification

Điều đó thật dễ dàng! Code này training SVC trên training set bằng cách sử dụng các lớp mục tiêu ban đầu từ 0 đến 9 (`y_train`), thay vì các lớp mục tiêu 5 so với phần còn lại (`y_train_5`). Sau đó, nó đưa ra một dự đoán (một dự đoán đúng trong trường hợp này). Dưới vỏ bọc, Scikit-Learn thực sự đã sử dụng chiến lược OvO: nó training 45 classifier nhị phân, lấy điểm quyết định cho hình ảnh và chọn ra lớp chiến thắng nhiều trận đấu tay đôi nhất.

Điểm cao nhất thực sự là điểm tương ứng với lớp 5:

Nếu bạn muốn buộc Scikit-Learn sử dụng một đấu một hoặc một đấu với phần còn lại, bạn có thể sử dụng các lớp `OneVsOneClassifier` hoặc `OneVsRestClassifier`. Chỉ cần tạo một thể hiện và chuyển một classifier cho hàm tạo của nó (nó thậm chí không cần phải là một classifier nhị phân). Ví dụ: code này tạo classifier nhiều lớp bằng cách sử dụng chiến lược OvR, dựa trên SVC:

Training `SGDClassifier` (hoặc `RandomForestClassifier`) cũng dễ dàng như vậy:

Lần này Scikit-Learn không phải chạy OvR hoặc OvO vì classifier SGD có thể classification

trực tiếp các individual thành nhiều lớp. Phương thức `decision_function()` hiện trả về một giá trị cho mỗi lớp. Hãy xem điểm mà classifier SGD gán cho từng lớp:

Bạn có thể thấy rằng classifier khá tự tin về dự đoán của nó: hầu hết tất cả các điểm phần lớn đều là âm, trong khi lớp 5 có số điểm là 2412,5. Model có chút nghi ngờ về lớp 3, đạt số điểm 573,5. Tất nhiên bây giờ bạn muốn đánh giá classifier này. Như thường lệ, bạn có thể sử dụng cross-validation. Sử dụng hàm `cross_val_score()` để đánh giá độ chính xác của `SGDClassifier`:

Nó đạt hơn 84% trong tất cả các lần thử nghiệm. Nếu bạn sử dụng classifier ngẫu nhiên, bạn sẽ đạt được độ chính xác 10%, vì vậy đây không phải là điểm quá tệ nhưng bạn vẫn có thể làm tốt hơn nhiều. Chỉ cần mở rộng quy mô đầu vào (như đã thảo luận trong Chương 2) sẽ tăng độ chính xác lên trên 89%:

1.12 Phân tích lỗi

Nếu đây là một dự án thực, bây giờ bạn sẽ làm theo các bước trong danh sách kiểm tra dự án Machine Learning của mình (xem Phụ lục B). Bạn sẽ khám phá các tùy chọn chuẩn bị dữ liệu, thử nhiều models (liệt kê những cái tốt nhất và tuning hyperparameters của chúng bằng `GridSearchCV`) và tự động hóa nhiều nhất có thể. Ở đây, chúng ta sẽ cho rằng bạn đã tìm thấy một model đầy hứa hẹn và bạn muốn tìm cách cải thiện nó. Một cách để làm điều này là phân tích các loại lỗi mà nó mắc phải.

102 | Chương 3: Classification

Đầu tiên, hãy nhìn vào confusion matrix. Bạn cần đưa ra dự đoán bằng hàm `cross_val_predict()`, sau đó gọi hàm `confusion_matrix()`, giống như bạn đã làm trước đó:

Đó là rất nhiều con số. Sẽ thuận tiện hơn khi xem hình ảnh đại diện của confusion matrix bằng cách sử dụng hàm `matshow()` của Matplotlib:

Confusion matrix này trông khá đẹp vì hầu hết các hình ảnh đều nằm trên đường chéo chính, có nghĩa là chúng đã được phân loại chính xác. Số 5 trông tối hơn một chút so với các chữ số khác, điều này có thể có nghĩa là có ít hình ảnh về số 5 hơn trong dataset hoặc classifier không hoạt động tốt trên số 5 như trên các chữ số khác. Trên thực tế, bạn có thể xác minh rằng cả hai đều đúng.

Hãy tập trung cốt truyện vào các lỗi. Trước tiên, bạn cần chia từng giá trị trong confusion matrix cho số lượng hình ảnh trong lớp tương ứng để có thể so sánh tỷ lệ lỗi thay vì số lượng lỗi tuyệt đối (điều này sẽ làm cho các lớp có nhiều trông xấu một cách không công bằng):

Điền vào đường chéo các số 0 để chỉ giữ lại các lỗi và vẽ kết quả:

Bạn có thể thấy rõ các loại lỗi mà classifier mắc phải. Hãy nhớ rằng các hàng đại diện cho các lớp thực tế, trong khi các cột đại diện cho các lớp được dự đoán. Cột dành cho lớp 8 khá sáng, điều này cho bạn biết rằng nhiều hình ảnh bị classification sai thành 8s. Tuy nhiên, hàng dành cho lớp 8 không đến nỗi tệ, cho bạn biết rằng số 8 thực tế nói chung được phân loại chính xác là số 8. Như bạn có thể thấy, confusion matrix không nhất thiết phải đối xứng. Bạn cũng có thể thấy số 3 và số 5 thường bị nhầm lẫn (theo cả hai hướng).

Việc phân tích confusion matrix thường cung cấp cho bạn thông tin chi tiết về các cách cải thiện classifier của mình. Nhìn vào biểu đồ này, có vẻ như bạn nên dành nỗ lực vào việc giảm số 8 sai. Ví dụ: bạn có thể cố gắng thu thập thêm dữ liệu training cho các chữ số trông giống số 8 (nhưng không phải) để classifier có thể học cách phân biệt chúng với số 8 thực. Hoặc bạn có thể thiết kế features mới có thể hỗ trợ classifier—ví dụ: viết thuật toán để đếm số vòng lặp đóng (ví dụ: 8 có hai, 6 có một, 5 không có). Hoặc bạn có thể xử lý trước hình ảnh (ví dụ: sử dụng Scikit-Image, Pillow hoặc OpenCV) để làm cho

một số mẫu, chẳng hạn như vòng khép kín, nổi bật hơn.

Phân tích các lỗi riêng lẻ cũng có thể là một cách hay để hiểu rõ hơn về những gì classifier của bạn đang thực hiện và lý do tại sao nó thất bại, nhưng việc này khó khăn và tốn thời gian hơn. Ví dụ: hãy vẽ các ví dụ về 3 và 5 (hàm `plot_digits()` chỉ sử dụng hàm `imshow()` của Matplotlib; xem Jupyter notebook của chương này để biết chi tiết):

104 | Chương 3: Classification

Hai khối 5×5 ở bên trái hiển thị các chữ số được phân loại là 3 và hai khối 5×5 ở bên phải hiển thị các hình ảnh được phân loại là 5. Một số chữ số mà classifier sai (ví dụ: ở khối dưới cùng bên trái và trên cùng bên phải) được viết tẹt đến mức ngay cả con người cũng gặp khó khăn khi classification chúng (ví dụ: số 5 ở hàng đầu tiên và cột thứ hai thực sự trông giống như số 3 được viết sai). Tuy nhiên, hầu hết các hình ảnh bị classification sai dường như là lỗi hiển nhiên đối với chúng ta và thật khó hiểu tại sao classifier lại mắc lỗi. Lý do là chúng ta đã sử dụng SGDClassifier đơn giản, là model tuyến tính. Tất cả những gì nó làm là gán weight cho mỗi lớp cho mỗi pixel và khi nhìn thấy một hình ảnh mới, nó chỉ tổng hợp cường độ pixel có weight để tính điểm cho mỗi lớp. Vì vậy, vì 3 giây và 5 giây chỉ khác nhau một vài pixel nên model này sẽ dễ khiến chúng nhầm lẫn.

Sự khác biệt chính giữa 3s và 5s là vị trí của đường nhỏ nối đường trên với cung dưới. Nếu bạn vẽ số 3 với điểm nối hơi dịch sang trái, classifier có thể classification nó thành số 5 và ngược lại. Nói cách khác, classifier này khá nhạy cảm với sự dịch chuyển và xoay hình ảnh. Vì vậy, một cách để giảm bớt sự nhầm lẫn 3/5 là xử lý trước các hình ảnh để đảm bảo rằng chúng được căn giữa và không bị xoay quá nhiều. Điều này có lẽ cũng sẽ giúp giảm các lỗi khác.

1.13 Classification đa nhãn

Cho đến nay mỗi phiên bản luôn được gán cho một lớp duy nhất. Trong một số trường hợp, bạn có thể muốn classifier của mình xuất ra nhiều lớp cho mỗi phiên bản. Hãy xem xét một classifier nhận dạng khuôn mặt: nó nên làm gì nếu nhận ra một số người trong cùng một bức ảnh? Nó sẽ đính kèm một thẻ cho mỗi người mà nó nhận ra. Giả sử classifier đã được training để nhận dạng ba khuôn mặt, Alice, Bob và Charlie. Sau đó, khi classifier hiển thị hình ảnh của Alice và Charlie, nó sẽ xuất ra `[1, 0, 1]` (có nghĩa là “Alice có, Bob không, Charlie có”). Hệ thống classification xuất ra nhiều thẻ nhị phân như vậy được gọi là hệ thống classification đa nhãn.

Chúng ta sẽ không đi sâu vào nhận dạng khuôn mặt mà hãy xem một ví dụ đơn giản hơn, chỉ nhằm mục đích minh họa:

Code này tạo một mảng `y_multilabel` chứa hai nhãn mục tiêu cho mỗi hình ảnh chữ số: nhãn đầu tiên cho biết chữ số đó có lớn hay không (7, 8 hoặc 9) và nhãn thứ hai cho biết nó có phải là số lẻ hay không. Các dòng tiếp theo tạo một phiên bản `KNeighborsClassifier` (hỗ trợ classification đa nhãn, mặc dù không phải tất cả các classifier đều làm được) và chúng ta training nó bằng cách sử dụng mảng nhiều mục tiêu. Bây giờ bạn có thể đưa ra dự đoán và nhận thấy rằng nó đưa ra hai nhãn:

Có nhiều cách để đánh giá classifier nhiều nhãn và việc chọn số liệu phù hợp thực sự phụ thuộc vào dự án của bạn. Một cách tiếp cận là đo điểm F1 cho từng nhãn riêng lẻ (hoặc bất kỳ số liệu classification nhị phân nào khác đã thảo luận trước đó), sau đó chỉ cần tính điểm trung bình. Code này tính điểm F1 trung bình trên tất cả các nhãn:

Điều này giả định rằng tất cả các nhãn đều quan trọng như nhau, tuy nhiên, điều này có thể không đúng. Đặc biệt, nếu bạn có nhiều ảnh về Alice hơn Bob hoặc Charlie, bạn

có thể muốn cho nhiều weight hơn vào điểm của classifier đối với các bức ảnh về Alice. Một lựa chọn đơn giản là gán cho mỗi nhãn một weight bằng với độ hỗ trợ của nó (tức là số lượng

106 | Chương 3: Classification

Trường hợp có nhãn mục tiêu đó). Để thực hiện việc này, chỉ cần đặt Average="weighted" trong code trước đó.4

1.14 Classification đa đầu ra

Loại nhiệm vụ classification cuối cùng mà chúng ta sẽ thảo luận ở đây được gọi là multioutput– multiclass classification (hoặc đơn giản là classification multioutput). Nó chỉ đơn giản là sự khái quát hóa của classification đa nhãn trong đó mỗi nhãn có thể là nhiều lớp (nghĩa là nó có thể có nhiều hơn hai giá trị có thể).

Để minh họa điều này, hãy xây dựng một hệ thống loại bỏ nhiễu khỏi hình ảnh. Nó sẽ lấy đầu vào là một hình ảnh chữ số nhiễu và (hy vọng) nó sẽ xuất ra một hình ảnh chữ số rõ ràng, được biểu thị dưới dạng một mảng cường độ điểm ảnh, giống như các hình ảnh MNIST. Lưu ý rằng đầu ra của trình được phân loại là nhiều nhãn (một nhãn cho mỗi pixel) và mỗi nhãn có thể có nhiều giá trị (cường độ pixel nằm trong khoảng từ 0 đến 255). Do đó, đây là một ví dụ về hệ thống classification nhiều đầu ra.

Ranh giới giữa classification và regression đôi khi bị mờ, chẳng hạn như trong ví dụ này. Có thể cho rằng, việc dự đoán cường độ điểm ảnh giống với regression hơn là classification. Hơn nữa, hệ thống nhiều đầu ra không bị giới hạn ở các tác vụ classification; bạn thậm chí có thể có một hệ thống xuất ra nhiều nhãn cho mỗi phiên bản, bao gồm cả nhãn lớp và nhãn giá trị.

Hãy bắt đầu bằng cách tạo tập training và test set bằng cách lấy hình ảnh MNIST và thêm nhiễu vào cường độ pixel của chúng bằng hàm randint() của NumPy. Hình ảnh mục tiêu sẽ là hình ảnh gốc:

Chúng ta hãy xem qua hình ảnh từ test set (vâng, chúng ta đang theo dõi dữ liệu thử nghiệm, vì vậy bạn nên cau mày ngay bây giờ):

Bên trái là hình ảnh đầu vào bị nhiễu và bên phải là hình ảnh mục tiêu sạch. Bây giờ, hãy training classifier và làm cho nó sạch hình ảnh này:

Có vẻ đủ gần mục tiêu! Điều này kết thúc chuyến tham quan classification của chúng ta. Bây giờ bạn đã biết cách chọn số liệu tốt cho các nhiệm vụ classification, chọn sự cân bằng giữa precision/recall thích hợp, so sánh các classifier và nói chung là xây dựng các hệ thống classification tốt cho nhiều nhiệm vụ khác nhau.

2 Bài tập

1. Cố gắng xây dựng classifier cho MNIST dataset để đạt được độ chính xác trên 97% trên test set. Gợi ý: KNeighborsClassifier hoạt động khá tốt cho nhiệm vụ này; bạn chỉ cần tìm các giá trị hyperparameter tốt (thử grid search trên weights và n_neighbors hyperparameters).

2. Viết hàm có thể dịch chuyển hình ảnh MNIST theo bất kỳ hướng nào (trái, phải, lên hoặc xuống) một pixel.5 Sau đó, với mỗi hình ảnh trong training set, hãy tạo bốn bản sao được dịch chuyển (mỗi bản một hướng) và thêm chúng vào training set. Cuối cùng, hãy training model tốt nhất của bạn trên training set mở rộng này và đo độ chính xác của nó trên test set. Bạn nên nhận thấy rằng model của bạn thậm chí còn hoạt động tốt

hơn bây giờ! Kỹ thuật phát triển training set nhân tạo này được gọi là boosting dữ liệu hoặc mở rộng training set.

108 | Chương 3: Classification

3. Xử lý tàu Titanic dataset. Một nơi tuyệt vời để bắt đầu là Kaggle.

4. Xây dựng classifier thư rác (một bài tập khó hơn):

- Tải xuống các ví dụ về spam và ham từ datasets công khai của Apache SpamAssassin.
- Giải nén datasets và làm quen với định dạng dữ liệu.
- Chia datasets thành training set và test set.
- Viết chuẩn bị dữ liệu pipeline để chuyển đổi từng email thành vector feature. Sự chuẩn bị của bạn pipeline sẽ chuyển đổi một email thành một vector (thưa thớt) cho biết sự hiện diện hay vắng mặt của từng từ có thể. Ví dụ: nếu tất cả các email chỉ chứa bốn từ, “Xin chào”, “như thế nào”, “là”, “bạn”, thì email “Xin chào bạn Xin chào Xin chào bạn” sẽ được chuyển đổi thành một vector [1, 0, 0, 1] (có nghĩa là [“Xin chào” có mặt, “làm thế nào” vắng mặt, “có” vắng mặt, “bạn” có mặt]) hoặc [3, 0, 0, 2] nếu bạn muốn đếm số lần xuất hiện của mỗi từ.

Bạn có thể muốn thêm hyperparameters vào pipeline chuẩn bị của mình để kiểm soát xem có loại bỏ tiêu đề email hay không, chuyển đổi từng email thành chữ thường, xóa dấu câu, thay thế tất cả URLs bằng “URL”, thay thế tất cả các số bằng “NUMBER” hoặc thậm chí thực hiện việc cắt gốc (tức là cắt bỏ phần cuối của từ; có sẵn các thư viện Python để thực hiện việc này).

Cuối cùng, hãy thử một số classifier và xem liệu bạn có thể xây dựng một classifier thư rác tuyệt vời với cả recall cao và precision cao hay không.

Bạn có thể tìm thấy giải pháp cho những bài tập này trong Jupyter notebooks tại <https://github.com/ageron/handson-ml2>.

2.1 Model training CHAPTER 4

Cho đến nay chúng ta đã xử lý Machine Learning models và các thuật toán training của chúng chủ yếu giống như black box. Nếu bạn đã xem qua một số bài tập trong các chương trước, bạn có thể ngạc nhiên về khối lượng công việc bạn có thể làm mà không cần biết bất kỳ điều gì về những gì ẩn giấu: bạn đã tối ưu hóa hệ thống regression, bạn đã cải thiện classifier hình ảnh chữ số và thậm chí bạn đã xây dựng một classifier thư rác từ đầu, tất cả những điều này mà không biết chúng thực sự hoạt động như thế nào. Thực sự, trong nhiều tình huống bạn không thực sự cần biết chi tiết triển khai.

Tuy nhiên, hiểu rõ về cách mọi thứ hoạt động có thể giúp bạn nhanh chóng sử dụng model thích hợp, thuật toán training phù hợp để sử dụng và một bộ hyperparameters phù hợp cho nhiệm vụ của bạn. Hiểu những gì ẩn sâu cũng sẽ giúp bạn gỡ lỗi các vấn đề và thực hiện phân tích lỗi hiệu quả hơn. Cuối cùng, hầu hết các chủ đề được thảo luận trong chương này sẽ rất cần thiết trong việc hiểu, xây dựng và training neural networks (được thảo luận trong Phần II của cuốn sách này).

Trong chương này, chúng ta sẽ bắt đầu bằng việc xem xét Linear Regression model, một trong những models đơn giản nhất hiện có. Chúng ta sẽ thảo luận về hai cách rất khác nhau để training nó:

- Sử dụng phương trình “dạng đóng” trực tiếp để tính toán trực tiếp các tham số model phù hợp nhất với model với training set (tức là model parameters giúp giảm thiểu hàm chi phí trên training set).

- Sử dụng phương pháp tối ưu hóa lặp lại được gọi là Gradient Descent (GD) để điều chỉnh dần model parameters để giảm thiểu hàm chi phí so với training set, cuối cùng hội

tụ về cùng một bộ parameters như phương pháp đầu tiên. Chúng ta sẽ xem xét một số biến thể của Gradient Descent mà chúng ta sẽ sử dụng đi sử dụng lại khi nghiên cứu neural networks trong Phần II: Batch GD, Mini-batch GD và Stochastic GD.

Tiếp theo chúng ta sẽ xem xét Polynomial Regression, một model phức tạp hơn có thể phù hợp với datasets phi tuyến tính. Vì model này có nhiều parameters hơn Linear Regression nên nó thiên về overfitting trong dữ liệu training hơn, vì vậy chúng ta sẽ xem xét cách phát hiện xem trường hợp này có xảy ra hay không bằng cách sử dụng các đường cong học tập và sau đó chúng ta sẽ xem xét một số kỹ thuật chính quy hóa có thể làm giảm rủi ro của overfitting đối với training set.

Cuối cùng, chúng ta sẽ xem xét thêm hai models thường được sử dụng cho các tác vụ classification: Logistic Regression và Softmax Regression.

Sẽ có khá nhiều phương trình toán học trong chương này, sử dụng các khái niệm cơ bản của đại số tuyến tính và phép tính. Để hiểu các phương trình này, bạn cần biết vector và matrix là gì; cách hoán vị, nhân và đảo ngược chúng; và đạo hàm riêng là gì. Nếu bạn không quen với những khái niệm này, vui lòng xem qua các hướng dẫn giới thiệu về đại số tuyến tính và phép tính có sẵn dưới dạng Jupyter notebooks trong tài liệu bổ sung trực tuyến. Đối với những người thực sự dị ứng với toán học, bạn vẫn nên đọc hết chương này và bỏ qua các phương trình; hy vọng bài viết sẽ đủ để giúp bạn hiểu hầu hết các khái niệm.

2.2 Regression tuyến tính

Trong Chương 1, chúng ta đã xem xét regression model đơn giản về sự hài lòng trong cuộc sống: $\text{life_satisfaction} = 0 + 1 \times \text{GDP_per_capita}$.

Model này chỉ là hàm tuyến tính của feature `GDP_per_capita` đầu vào. 0 và 1 là parameters của model.

Tổng quát hơn, model tuyến tính đưa ra dự đoán bằng cách tính tổng weight của features đầu vào, cộng với một hằng số được gọi là số hạng bias (còn gọi là số hạng chặn), như được hiển thị trong Công thức 4-1.

Phương trình 4-1. Dự đoán Linear Regression model $y = 0 + 1x_1 + 2x_2 + \dots + nx_n$

Trong phương trình này:

- $\hat{y}$ là giá trị dự đoán.
- n là số features.
- x_i là giá trị feature thứ i .
- j là model parameter thứ j (bao gồm số hạng bias 0 và feature weights 1, 2, ..., n).

112 | Chương 4: Training Models

Điều này có thể được viết chính xác hơn nhiều bằng cách sử dụng dạng vector hóa, như thể hiện trong phương trình 4-2.

Phương trình 4-2. Dự đoán Linear Regression model (dạng vector hóa) $y = h \cdot x = 0 + 1x_1 + 2x_2 + \dots + nx_n$

Trong phương trình này:

- $\mathbf{h}$ là vector parameter của model, chứa số hạng bias 0 và feature weights 1 đến n .
- $\mathbf{x}$ là vector feature của thể hiện, chứa x_0 đến x_n , với x_0 luôn bằng 1.
- $\mathbf{h} \cdot \mathbf{x}$ là tích vô hướng của vector $\mathbf{h}$ và $\mathbf{x}$, tất nhiên bằng $0x_0 + 1x_1 + 2x_2 + \dots + nx_n$.
- h là hàm giả thuyết, sử dụng model parameters $\mathbf{h}$.

Trong Machine Learning, vector thường được biểu diễn dưới dạng vector cột, là mảng 2D có một cột duy nhất. Nếu $\mathbf{w}$ và $\mathbf{x}$ là vector cột, thì dự đoán là $y = \mathbf{w}^T \mathbf{x}$, trong đó $\mathbf{w}^T$ là chuyển vị của $\mathbf{w}$ (vector hàng thay vì vector cột) và $\mathbf{w} \mathbf{x}$ là phép nhân matrix của $\mathbf{w}$ và $\mathbf{x}$. Tất nhiên đó là cùng một dự đoán, ngoại trừ việc bây giờ nó được biểu diễn dưới dạng matrix một ô thay vì giá trị vô hướng. Trong cuốn sách này, tôi sẽ sử dụng ký hiệu này để tránh chuyển đổi giữa tích số chấm và phép nhân matrix.

OK, đó là Linear Regression model—nhưng chúng ta training nó như thế nào? Chà, recall rằng việc training model có nghĩa là thiết lập parameters của nó sao cho model phù hợp nhất với training set. Với mục đích này, trước tiên chúng ta cần thước đo mức độ phù hợp (hoặc kém) của model với dữ liệu training. Trong Chương 2, chúng ta đã thấy rằng thước đo hiệu suất phổ biến nhất của regression model là Sai số bình phương trung bình gốc (RMSE) (Phương trình 2-1). Do đó, để training Linear Regression model, chúng ta cần tìm giá trị của $\mathbf{w}$ làm giảm thiểu RMSE. Trong thực tế, việc cực tiểu hóa mean squared error (MSE) đơn giản hơn so với RMSE và dẫn đến kết quả tương tự (vì giá trị cực tiểu hóa một hàm cũng cực tiểu hóa căn bậc hai của nó).¹

MSE của giả thuyết Linear Regression h trên training set X được tính bằng Công thức 4-3.

Phương trình 4-3. Hàm chi phí MSE cho Linear Regression model

$$MSE(X, h) = \frac{1}{m} \sum_{i=1}^m (x_i - y_i)^2$$

Hầu hết các ký hiệu này đã được trình bày trong Chương 2 (xem “Các ký hiệu” ở trang 40). Sự khác biệt duy nhất là chúng ta viết h thay vì chỉ $\mathbf{w}$ để làm rõ rằng model được tham số hóa bởi vector $\mathbf{w}$. Để đơn giản hóa các ký hiệu, chúng ta sẽ chỉ viết $MSE(X, h)$ thay vì $MSE(X, \mathbf{w})$.

2.3 Phương trình chuẩn

Để tìm giá trị của $\mathbf{w}$ làm tối thiểu hóa hàm chi phí, có một nghiệm dạng đóng — nói cách khác, một phương trình toán học cho kết quả trực tiếp. Đây được gọi là Phương trình chuẩn (Phương trình 4-4).

Phương trình 4-4. Phương trình chuẩn
$$X^T X \mathbf{w} = X^T \mathbf{y}$$

Trong phương trình này:

- $\mathbf{w}$ là giá trị của $\mathbf{w}$ làm hàm chi phí cực tiểu.
- $\mathbf{y}$ là vector của các giá trị đích chứa $y(1)$ đến $y(m)$.

Hãy tạo một số dữ liệu tuyến tính để kiểm tra phương trình này (Hình 4-1):

114 | Chương 4: Training Models

Bây giờ hãy tính $\mathbf{w}$ bằng phương trình chuẩn. Chúng ta sẽ sử dụng hàm `inv()` từ mô-đun đại số tuyến tính của NumPy (`np.linalg`) để tính nghịch đảo của matrix và phương thức `dot()` để nhân matrix:

Hàm mà chúng ta sử dụng để tạo dữ liệu là $y = 4 + 3x_1 + \text{nhiều Gaussian}$. Hãy xem phương trình tìm thấy gì:

Chúng ta đã hy vọng $w_0 = 4$ và $w_1 = 3$ thay vì $w_0 = 4,215$ và $w_1 = 2,770$. Đủ gần, nhưng tiếng ồn khiến không thể khôi phục chính xác parameters của chức năng ban đầu.

Bây giờ chúng ta có thể đưa ra dự đoán bằng cách sử dụng $\mathbf{w}$:

Hãy vẽ đồ thị dự đoán của model này (Hình 4-2):

Thực hiện Linear Regression bằng Scikit-Learn rất đơn giản:²

116 | Chương 4: Training Models

Bản thân pseudoinverse được tính toán bằng cách sử dụng kỹ thuật phân tích nhân tử matrix tiêu chuẩn được gọi là Phân tích giá trị đơn (SVD) có thể phân tích matrix training set X thành phép nhân matrix của ba matrix $U \Sigma V$ (xem `numpy.linalg.svd()`). Giả nghịch đảo được tính là $X^+ = V \Sigma^+ U$. Để tính matrix Σ^+ , thuật toán lấy Σ và đặt về 0 tất cả các giá trị nhỏ hơn một giá trị ngưỡng nhỏ, sau đó nó thay thế tất cả các giá trị khác 0 bằng nghịch đảo của chúng và cuối cùng nó hoán vị matrix kết quả. Cách tiếp cận này hiệu quả hơn việc tính toán Phương trình chuẩn, hơn nữa nó xử lý tốt các trường hợp biên: thực sự, Phương trình chuẩn có thể không hoạt động nếu matrix $X^T X$ không khả nghịch (tức là số ít), chẳng hạn như nếu $m < n$ hoặc nếu một số features dư thừa, nhưng giả nghịch đảo luôn được xác định.

2.4 Độ phức tạp tính toán

Phương trình Chuẩn tính nghịch đảo của $X^T X$, là matrix $(n + 1) \times (n + 1)$ (trong đó n là số features). Độ phức tạp tính toán của việc đảo ngược matrix như vậy thường là từ $O(n^2.4)$ đến $O(n^3)$, tùy thuộc vào cách triển khai. Nói cách khác, nếu bạn nhân đôi số features, bạn sẽ nhân thời gian tính toán với khoảng $2^{2.4} = 5.3$ đến $2^3 = 8$.

Ngoài ra, khi bạn đã training Linear Regression model của mình (sử dụng Phương trình chuẩn hoặc bất kỳ thuật toán nào khác), các dự đoán sẽ rất nhanh: độ phức tạp tính toán là tuyến tính đối với cả số lượng phiên bản bạn muốn đưa ra dự đoán và số lượng features. Nói cách khác, việc đưa ra dự đoán trên số lượng phiên bản nhiều gấp đôi (hoặc gấp đôi số features) sẽ mất khoảng thời gian gần gấp đôi.

Bây giờ chúng ta sẽ xem xét một cách rất khác để training Linear Regression model, cách này phù hợp hơn với các trường hợp có số lượng lớn features hoặc có quá nhiều trường hợp training để vừa với bộ nhớ.

2.5 Gradient Descent

Gradient Descent là một thuật toán tối ưu hóa chung có khả năng tìm ra giải pháp tối ưu cho nhiều vấn đề. Ý tưởng chung của Gradient Descent là điều chỉnh parameters lặp đi lặp lại để giảm thiểu hàm chi phí.

Giả sử bạn bị lạc trong núi trong sương mù dày đặc và bạn chỉ có thể cảm nhận được gradient của mặt đất dưới chân mình. Một chiến lược tốt để nhanh chóng xuống đáy thung lũng là descent dốc theo hướng có gradient cao nhất. Đây chính xác là những gì Gradient Descent thực hiện: nó đo gradient cục bộ của hàm lỗi đối với vector parameter và nó đi theo hướng gradient giảm dần. Khi gradient bằng 0, bạn đã đạt đến mức tối thiểu!

Cụ thể, bạn bắt đầu bằng cách điền vào các giá trị ngẫu nhiên (điều này được gọi là khởi tạo ngẫu nhiên). Sau đó, bạn cải thiện nó dần dần, thực hiện từng bước nhỏ một, mỗi bước cố gắng giảm hàm chi phí (ví dụ: MSE), cho đến khi thuật toán hội tụ đến mức tối thiểu (xem Hình 4-3).

Một parameter quan trọng trong Gradient Descent là kích thước của các bước, được xác định bởi learning rate hyperparameter. Nếu learning rate quá nhỏ thì thuật toán sẽ phải trải qua nhiều lần lặp để hội tụ, việc này sẽ mất nhiều thời gian (xem Hình 4-4).

118 | Chương 4: Training Models

Mặt khác, nếu learning rate quá cao, bạn có thể nhảy qua thung lũng và đến phía bên kia, thậm chí có thể cao hơn mức bạn trước đó. Điều này có thể làm cho thuật toán phân kỳ, với các giá trị ngày càng lớn, không tìm được giải pháp tốt (xem Hình 4-5).

Cuối cùng, không phải tất cả các hàm chi phí đều trông giống như những chiếc bát thông thường, đẹp đẽ. Có thể có những hố, những rặng núi, những cao nguyên và đủ loại địa hình không đều, khiến cho việc hội tụ trở nên khó khăn ở mức tối thiểu. Hình 4-6 cho thấy hai thách thức chính với Gradient Descent. Nếu việc khởi tạo ngẫu nhiên bắt đầu thuật toán ở bên trái thì nó sẽ hội tụ về mức tối thiểu cục bộ, không tốt bằng mức tối thiểu toàn cục. Nếu xuất phát từ bên phải thì sẽ mất rất nhiều thời gian để vượt qua cao nguyên. Và nếu bạn dừng lại quá sớm, bạn sẽ không bao giờ đạt được mức tối thiểu toàn cầu.

May mắn thay, hàm chi phí MSE cho Linear Regression model lại là một hàm lồi, có nghĩa là nếu bạn chọn bất kỳ hai điểm nào trên đường cong, đoạn đường nối chúng sẽ không bao giờ cắt đường cong. Điều này ngụ ý rằng không có cực tiểu cục bộ, chỉ có một cực tiểu toàn cầu. Nó cũng là một hàm liên tục có gradient không bao giờ thay đổi đột ngột.³ Hai sự thật này có một hệ quả lớn: Gradient Descent được đảm bảo tiếp cận mức đóng tối thiểu toàn cục một cách tùy ý (nếu bạn đợi đủ lâu và nếu learning rate không quá cao).

Trên thực tế, hàm giá có hình dạng của một cái bát, nhưng nó có thể là một cái bát thon dài nếu features có tỷ lệ rất khác nhau. Hình 4-7 hiển thị Gradient Descent trên tập training trong đó features 1 và 2 có cùng tỷ lệ (ở bên trái) và trên training set trong đó feature 1 có giá trị nhỏ hơn nhiều so với feature 2 (ở bên phải).⁴

120 | Chương 4: Training Models